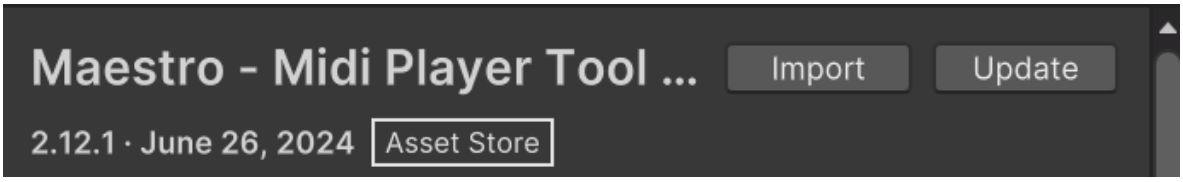
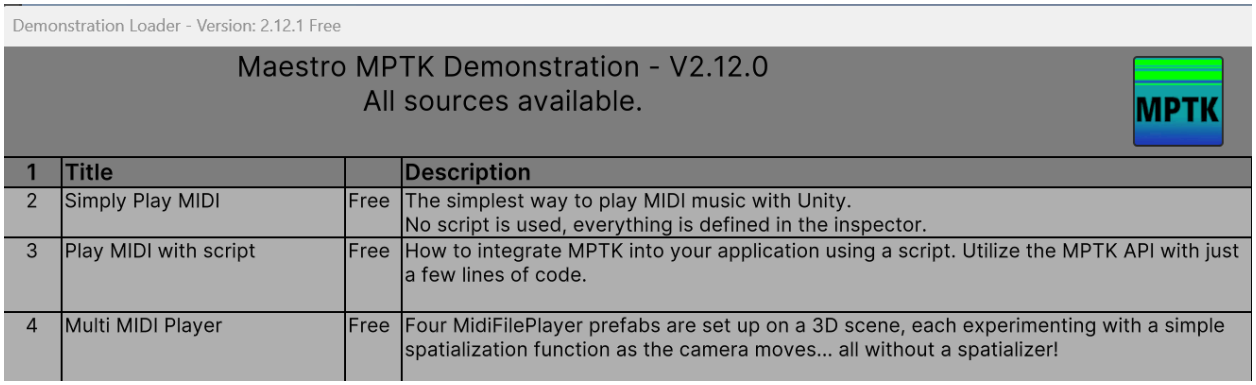
Isochord Visualization Tutorial

This project uses the Maestro MIDI Player Tool Kit from the Unity Asset Store. Before starting this tutorial, please add Maestro to your Unity account and click the “Open in Unity” button **while a new Unity project is open** to import the asset into the project.



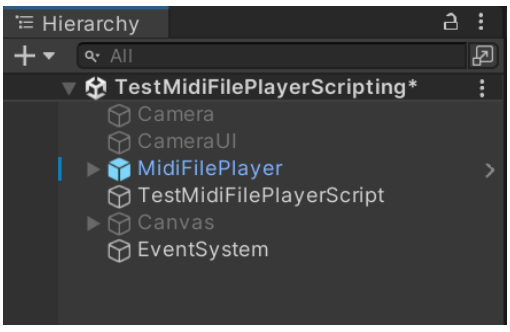
1. Open the “Maestro” dropdown menu in the top toolbar and select “Load Demonstration”.
2. Select the second demo in the list, titled “Play MIDI with script”.



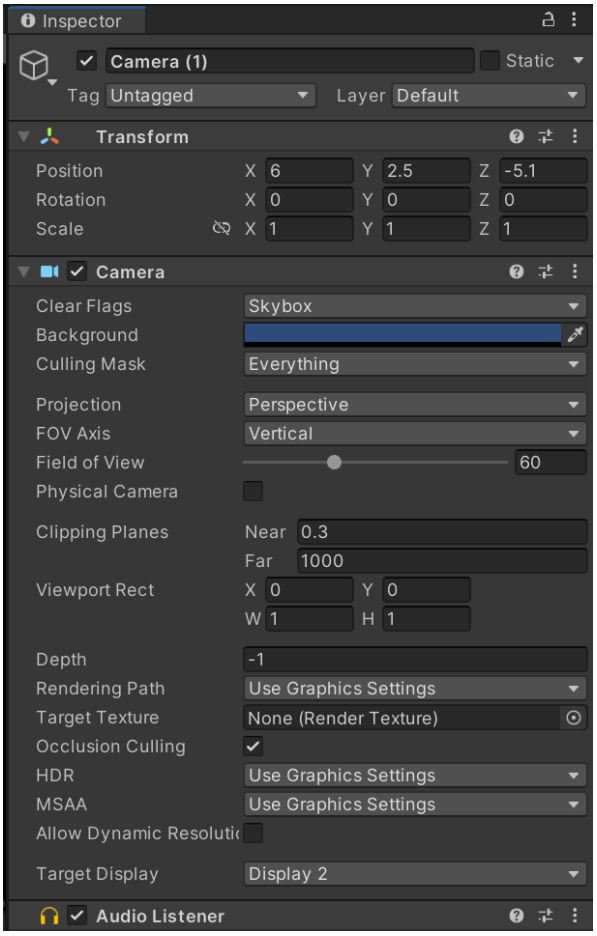
1	Title		Description
2	Simply Play MIDI	Free	The simplest way to play MIDI music with Unity. No script is used, everything is defined in the inspector.
3	Play MIDI with script	Free	How to integrate MPTK into your application using a script. Utilize the MPTK API with just a few lines of code.
4	Multi MIDI Player	Free	Four MidiFilePlayer prefabs are set up on a 3D scene, each experimenting with a simple spatialization function as the camera moves... all without a spatializer!

This demo creates a GUI that allows us to change parameters of MIDI files as they play in real time. However, we do not need this GUI for the purposes of our visualization, so we will remove a few of the objects in our hierarchy.

3. Uncheck the Camera, CameraUI, and Canvas objects; they should now appear darker than the other objects in the hierarchy.



4. Create a new Camera, and change its settings to those in the following screenshot. Make sure that the camera is targeting Display 2.
   CORRECTION: Change the -5.1 Z-value to -5.4; this improves grid visibility in the Game window.

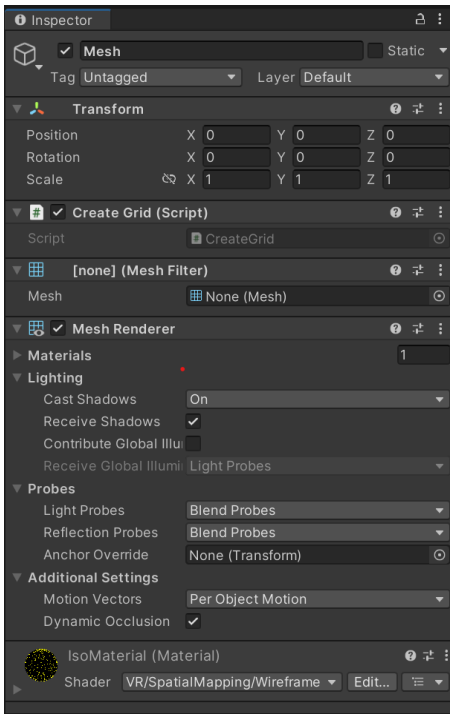


5. At the top of the Game window, change the target display to Display 2.
6. Create a new empty GameObject and name it “Mesh”.
7. Create a new Script and attach it to the Mesh. Then, open the Script. This will be where we write all of the code for this project.
8. Add the following line to the Script, just above the class declaration.

```
using UnityEngine;

[RequireComponent(typeof(MeshFilter), requiredComponent2:typeof(MeshRenderer))]
No asset usages
public class CreateGrid : MonoBehaviour
{
```

9. Run the Unity project; new properties should appear in the Inspector window for the Mesh GameObject.
10. Add the following Material to the Mesh; this will allow us to see the lines as they are drawn between grid vertices. This is done by creating a new Material (I named mine IsoMaterial) and attaching it to the Mesh object. Then, select the Shader dropdown and search for VR/SpatialMapping/Wireframe.



11. Return to the Script. Change the structure so that it resembles the screenshot below; remove the Update function, and create a function called Generate, which is called by the Start function.

```
public class CreateGrid : MonoBehaviour
{
    1 usage
    public void Generate()
    {

    }

    Event function
    void Start()
    {
        Generate();
    }
}
```

12. Add the following dependencies to the start of the Script, and define the variables midiFilePlayer, vertices, and mesh at the top of the body of the class.

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEditor;
using UnityEngine.Rendering.VirtualTexturi
using System;
using MidiPlayerTK;
using MPTK.NAudio.Midi;
using UnityEngine.UIElements;

[RequireComponent(typeof(MeshFilter), requi
☢ No asset usages
public class CreateGrid : MonoBehaviour
{
    MidiFilePlayer midiFilePlayer;
    private Vector3[] vertices;
    private Mesh mesh;
```

13. Add the following code to the Generate function.

```
public void Generate()
{
    midiFilePlayer = FindObjectOfType<MidiFilePlayer>();
    midiFilePlayer.OnEventNotesMidi.AddListener(NotesToPlay);

    GetComponent<MeshFilter>().mesh = mesh = new Mesh();
    mesh.name = "Procedural Grid";

    vertices = new Vector3[12 * 6];
    Vector2[] uv = new Vector2[vertices.Length];
    Vector4[] tangents = new Vector4[vertices.Length];
    Vector4 tangent = new Vector4(1f, 0f, 0f, w:-1f);
}
```

14. Add the following for loops to the Generate function, just beneath the code from the last step; these will construct the offset rows of the grid. Then, add the code in the subsequent screenshot (in the Generate function, just below the loops) to instantiate the objects and create the grid.

```
22          for (int i = 0, x = 0; i < 12; i++, x++)
21          {
20              vertices[i] = new Vector3(x, 0);
19              vertices[i + 12] = new Vector3(x + 0.5f, 1);
18          }
17          for (int i = 24, x = 0; i < 36; i++, x++)
16          {
15              vertices[i] = new Vector3(x, 2);
14              vertices[i + 12] = new Vector3(x + 0.5f, 3);
13          }
12          for (int i = 48, x = 0; i < 60; i++, x++)
11          {
10              vertices[i] = new Vector3(x, 4);
 9              vertices[i + 12] = new Vector3(x + 0.5f, 5);
 8          }
```

```
 6          mesh.vertices = vertices;
 5          mesh.uv = uv;
 4          mesh.tangents = tangents;
 3
 2          int[] triangles = new int[12 * 6 * 6];
 1          mesh.triangles = triangles;
54          mesh.RecalculateNormals();
```

15. Beneath the code written in the last step, create a new function inside Generate called NotesToPlay; populate the function with the code in the screenshot below. The following code checks whether a MIDI event has occurred; if it has, then a message is printed to the console displaying the MIDI information, and a sphere is instantiated on the grid at the position of the MIDI event's note value.

```
11      mesh.RecalculateNormals();
10
9       void NotesToPlay(List<MPTKEvent> mptkEvents)
8       {
7           foreach (MPTKEvent mptkEvent in mptkEvents)
6           {
5               if (mptkEvent.Command == MPTKCommand.NoteOn)
4               {
3                   Debug.Log(
2                       $"Note on Time:{mptkEvent.RealTime} millisecond " +
1                       $"Note:{mptkEvent.Value}  Duration:{mptkEvent.Duration}" +
67                      $"millisecond  Velocity:{mptkEvent.Velocity}");
1
2                   GameObject sphere = GameObject.CreatePrimitive(PrimitiveType.Sphere);
3                   sphere.transform.localScale -= new Vector3(0.6f, 0.6f, 0.6f);
4                   var sphereRenderer = sphere.GetComponent<Renderer>();
5                   sphereRenderer.material.SetColor(name:"_Color", Color.blue);
```

16. Inside the NotesToPlay function, directly underneath the code written in the last step, write the following switch statement, which determines the note value of the MIDI event and moves the sphere to the correct position on the isochord grid.

```
switch (mptkEvent.Value % 12)
{
    case 0:
        sphere.transform.position = vertices[6];
        break;
    case 1:
        sphere.transform.position = vertices[1];
        break;
    case 2:
        sphere.transform.position = vertices[8];
        break;
    case 3:
        sphere.transform.position = vertices[3];
        break;
```

```
case 4:
    sphere.transform.position = vertices[10];
    break;
case 5:
    sphere.transform.position = vertices[5];
    break;
case 6:
    sphere.transform.position = vertices[20];
    break;
case 7:
    sphere.transform.position = vertices[7];
    break;
```

```
    case 8:
        sphere.transform.position = vertices[2];
        break;
    case 9:
        sphere.transform.position = vertices[9];
        break;
    case 10:
        sphere.transform.position = vertices[4];
        break;
    case 11:
        sphere.transform.position = vertices[11];
        break;
}
```

17. After the switch statement, add this Destroy function, which removes the sphere after the full duration of the MIDI event.

```
                    break;
                }

                Destroy(sphere, t:mptkEvent.Duration / 1000f);
            }
        }
    }
}
```

18. Finally, add a function called OnDrawGizmos, outside of the Generate function, directly before the Start function; this is what allows us to see the grid vertices (called Gizmos) in the Game window.

```
    private void OnDrawGizmos()
    {
        if (vertices == null)
        {
            return;
        }
        Gizmos.color = Color.white;
        for (int i = 0; i < vertices.Length; i++)
        {
            Gizmos.DrawSphere(vertices[i], radius: 0.1f);
        }
    }

    Event function
    void Start()
    {
        Generate();
    }
}
```

19. The Script is complete! Now we can go back to the Unity project and press play. A grid should appear in the Game window, and spheres should appear at the correct locations when MIDI events are triggered. A log of all the MIDI events should also be displayed in the console in real time.

